Mprofit Deep-Dive: Product & Technology Case Study

Compiled for internal reference — building a next-gen wealth management platform Based on publicly available information only (product pages, help docs, published third-party license disclosures, job/funding data, user reviews). No proprietary source code, internal APIs, or non-public systems were accessed.

---

1. Company & Business Snapshot

Founded	2009, Mumbai
Founders	Kiran Shah (CEO), Pallav Bhatt (CTO), Manish Jain (VP Biz Dev)
Team size	~11–50 employees
Funding	~$2M raised from Zerodha's Rainmatter, Enam Holdings, Gruhas Proptech, AuXano Capital — largely bootstrapped before that
Users	200,000+ signups, presence in 300+ Indian cities
Core claim	Largest data-import coverage in India — 700–1,600+ institutions depending on the year, 3,000–5,000+ file formats
Segments served	Retail investors, HNIs, financial advisors, brokers, family offices, CAs

Positioning in one line: Mprofit doesn't compete on advisory or trading — it competes on being the most reliable *aggregation and accounting* layer for multi-asset Indian portfolios, then monetizing that via tiered SaaS pricing to investors and wealth professionals.

---

2. Confirmed Technology Stack

This section is built directly from Mprofit's own **Third-Party Licenses** disclosure page (a legally-required public document listing OSS/third-party dependencies) — this is the most reliable technical signal available since it comes straight from them, not inference.

2.1 Desktop Application (Windows)

- **Runtime**: Microsoft .NET Framework 2.0 → strongly suggests a **C# / WinForms** desktop application, likely with a codebase dating back to the 2009–2012 era that's been incrementally maintained rather than rewritten.

- **Reporting**: Microsoft Report Viewer 2005 SP1 (RDLC-style embedded reporting) — this is almost certainly how the PDF/Excel report generation (capital gains, XIRR, holdings) is rendered on desktop.

- **Local data store**: **SQLite** — confirms the desktop app is a local, file-based, serverless application (each user's portfolio data lives in a local SQLite file), not something that round-trips to a server for every read. This explains why the desktop product could historically run fully offline.

- **Document parsing pipeline** (the real IP core): a *stack* of PDF/Excel tools rather than one library —

    - Winnovative PDF-to-Text Converter for .NET
    - xpdf's `pdftotext`
    - QPDF (PDF transformation/repair)
    - Bytescout Spreadsheet SDK (Excel parsing/generation)
    - 7-Zip (handling compressed statement bundles)
    - Newtonsoft.Json (data interchange)

    Inference: their "5,000+ format" import claim is achieved not by one universal parser but by layering multiple PDF-text-extraction engines plus a large library of per-broker/per-RTA parsing templates/rules built on top. This is labor-intensive, format-fragile IP — consistent with the G2 review complaint about brokers requiring "confusing modifications" to import cleanly. It's a rules-engine + template-library approach, not an ML-based universal parser (at least as of the last stack update reflected in their license list).

2.2 Web / Cloud Application

- **Framework**: Angular 6 (`@angular/core`, `cdk`, `flex-layout`, `router`, `forms`) — a full Angular SPA, not React, not server-rendered.
- **UI**: Bootstrap 3 + jQuery (still present alongside Angular — common in Angular apps migrated from older jQuery-based UIs), `ngx-bootstrap`, custom modal/scroll/context-menu libraries.
- **Charts**: `@swimlane/ngx-charts` — their portfolio allocation and performance visualizations are built on this.
- **Data handling**: RxJS 6, Lodash, Moment.js, `big.js` (arbitrary-precision decimal math — **important**: this signals they deliberately avoid native JS floating-point for money/quantity calculations, which is the correct practice for financial software and worth replicating).
- **Document viewing**: `ng2-pdf-viewer` — used to show statement PDFs in-browser during import review/verification.
- **Session handling**: `@ng-idle` — auto-logout on inactivity, a basic but necessary fintech security control.
- **ID handling**: `hashids` — used to obfuscate sequential database IDs in URLs/API responses (a lightweight, non-cryptographic obfuscation technique, not true security — worth doing better in a rebuild).

Inference: the cloud web app was built (or last had a major dependency refresh) around 2018–2019 (Angular 6 shipped mid-2018). A modern rebuild would reasonably jump straight to Angular 17+/React or a metaframework rather than replicate this generation.

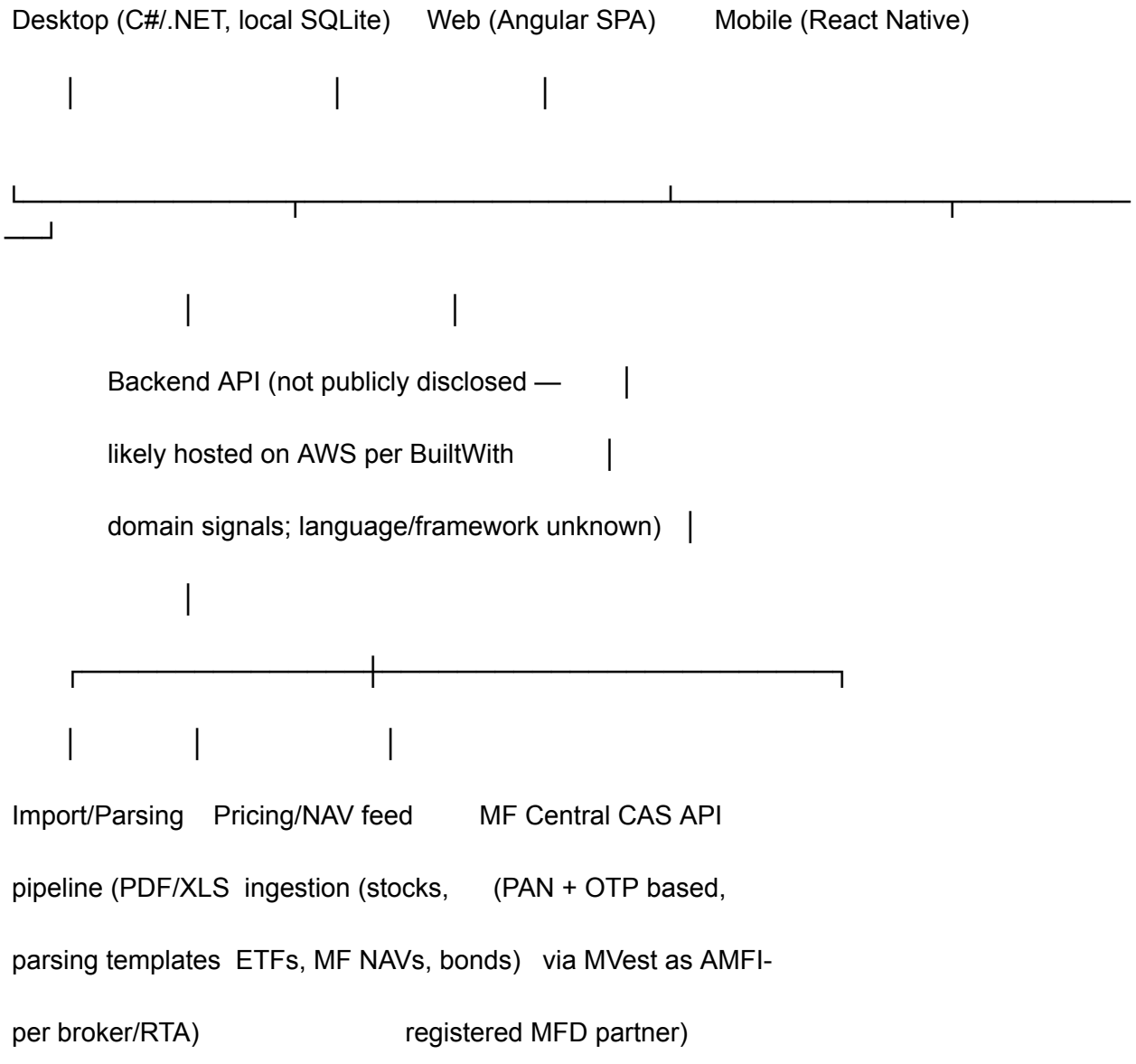
2.3 Mobile Apps (Android/iOS)

- **Framework**: **React Native 0.57.5** (2018-era) — single codebase for both platforms, not native Swift/Kotlin.
- **State management**: Redux + Redux-Saga (side-effect/async orchestration) + Redux-Persist, with two persistence backends:
  - `redux-persist-sensitive-storage` — encrypted/secure storage, almost certainly for auth tokens and PII
  - `redux-persist-filesystem-storage` — larger cached data (likely portfolio/report data) kept off the small AsyncStorage key-value limit
- **Security-relevant packages**:
  - `react-native-touch-id` → Face ID / Touch ID support (matches their App Store description)
  - `react-native-sensitive-info` → secure keychain/keystore-backed storage
  - `react-native-bcrypt` → local hashing, consistent with a device-level passcode that's verified **on-device** rather than sent to a server each time — a good pattern for offline PIN unlock.

- **Charts**: `victory-native`.
- **Navigation**: `react-navigation` v3.
- **Push**: `react-native-push-notification` (matches their stated lock-in/maturity-date alerts).

Inference: like the web app, the mobile stack reflects a 2018-era React Native setup. Given RN's major architecture changes since (Fabric/TurboModules, New Architecture), a new build should not mirror this version but the *feature and security pattern set* (biometric unlock, secure token storage, local PIN hash, offline-first caching) is exactly right to replicate.

2.4 Architectural Picture (composited)



The backend service layer itself is **not disclosed anywhere publicly** — no backend language, database engine (server-side), or infra-as-code details exist in any public source. This is the one part of their stack you cannot legitimately learn more about, and any further "reconstruction" attempt would cross from research into reverse-engineering, which I'd continue to decline to help with.

---

3. The Import Engine — Their Actual Moat

This deserves its own section because it's clearly Mprofit's core differentiator, not a side feature.

What it does: ingests contract notes, MF CAS statements, bank statements, e-NPS statements, and back-office broker files, across PDF/Excel/HTML/CSV/TXT/DBF, from 700–1,600+ distinct institutions, and normalizes them into a single transaction ledger.

How it's built (per the license evidence): a layered PDF/Excel extraction toolchain (Winnovative + xpdf + QPDF + Bytescout) feeding what's almost certainly a large, hand-maintained **library of per-institution parsing templates/rules** — because generic PDF-to-text extraction alone cannot handle the wildly inconsistent statement layouts across 700+ Indian brokers and banks. This is corroborated by the recurring user complaint that some broker statements need manual Excel template tweaks to import cleanly — a hallmark of a template-matching system hitting its edge cases, not a robust universal parser.

Where the API path is real and different: the **MF Central CAS API** integration is a genuinely modern, structured-data path (PAN + OTP-authenticated, redirect-based, likely SOAP/REST behind MF Central's infrastructure) that bypasses PDF parsing entirely for mutual funds — done in partnership with an AMFI-registered distributor (MVest) rather than built solo. Notably, Mprofit does **not** appear to have equivalent structured-API access for stock broker data (CDSL/NSDL-level API access is far more restrictive in India), which is why the PDF/Excel parsing pipeline remains necessary for the bulk of their equity-side imports.

Opportunity for a better version: this is genuinely the area with the most room to leapfrog —

- OCR + LLM-assisted statement parsing (vs. rigid per-broker templates) could dramatically cut the "700 institutions but constant edge-case breakage" problem.
- Wider structured-API coverage: CAMS/KFintech RTA APIs, CDSL Easi/Easiest APIs, Account Aggregator (AA) framework (India's RBI-regulated consent-based financial data sharing, live since 2021) — Mprofit's public materials don't mention AA integration at all, which is a significant gap for a 2026-built competitor, since AA gives consent-based, real-time, structured access to bank/MF/insurance/pension data without any PDF parsing.

---

4. Feature Inventory (for parity/gap planning)

Asset classes covered: Stocks, F&O, Mutual Funds, Bonds (traded), Fixed Deposits, NPS, PMS, AIF, ULIPs, PPF, Property, Gold/Silver.

Core capabilities:

- Multi-portfolio, multi-family consolidation (individual + family + group views)
- Auto price updates: stocks/ETFs (15-min delayed), daily MF NAV, daily bond prices
- Corporate action automation (dividends, bonus, splits, mergers, demergers) applied in bulk across portfolios
- XIRR / annualized return, absolute gain
- Capital gains reports: LTCG/STCG, with/without indexation, LTCG grandfathering (2018 rule), InvIT/REIT support
- Tax-software-compatible export formats (ClearTax, Winman, CompuTax)
- Historical month-end valuations
- Income reports, due-date reports (maturity, lock-in expiry), holding-period reports
- Simplified double-entry accounting module (P&L, balance sheet, trial balance) layered on top of the portfolio ledger — this is a differentiator vs. pure trackers like Sharesight
- Role-based multi-user access for advisors managing client portfolios
- Web + Windows desktop + Android + iOS parity (data shared via cloud sync)

Where reviews say it's weak (your whitespace):

- Import reliability on the long tail of brokers (manual template fixes needed)
- Support latency on non-trivial queries
- UI described as "tricky" for non-power-users
- No mention anywhere of Account Aggregator integration, open banking, or modern consent-based data sharing
- No visible AI/analytics layer (portfolio insights are historical/reporting-based, not advisory or predictive)

---

5. Pricing Model (for reference)

Two tracks — **Investors** and **Wealth Professionals** — replacing a legacy per-portfolio-count model (₹3,000–7,500/year) with tiered annual SaaS plans: Free → Lite (~~$33) → Plus (~~$83) → HNI (~$250), plus custom/enterprise pricing for advisors and CAs managing many client portfolios.

---

6. Recommendations if Building a "Deeper, Better" Version

1. **Modernize the import pipeline first** — this is the actual product, not a feature. Consider AA-framework integration as a first-class citizen (not an afterthought), supplemented by AI-assisted statement parsing (vision-LLM or fine-tuned document-extraction model) for the long tail PDF/Excel cases, with human-reviewable low-confidence flagging rather than silent failure.
2. **Decimal-safe math from day one** — their use of `big.js` on the JS side is a correct pattern; enforce fixed-point/decimal types end-to-end (including backend and DB schema) rather than floats, given this is financial data.
3. **Security posture to match/exceed**: biometric unlock + secure keystore storage + on-device PIN verification (their mobile pattern) is solid — pair it with proper JWT/OAuth session handling and real encryption at rest, not just ID obfuscation (their `hashids` approach is not a substitute for authorization checks).
4. **Structured-data-first architecture**: treat PDF parsing as the fallback path, not the primary path — prioritize every available official API (MF Central, AA framework, CAMS/KFintech, broker APIs like Zerodha Kite/Upstox where available) and only fall back to document parsing for asset classes with no API coverage (PMS, AIF, physical FDs, property).
5. **Advisory/insight layer as differentiation**: Mprofit is purely descriptive (what happened) — a "deeper" version could add prescriptive analytics (rebalancing suggestions, tax-loss harvesting flags, goal-tracking with course-correction nudges) without becoming a registered advisor, by keeping outputs informational rather than personalized recommendations (or by pursuing RIA registration deliberately, as PowerUp Money has done).
6. **Support/UX gap** is a real, cheap win — most reviews cite friction, not missing features. A significantly better onboarding/import-verification UX (clear diffs, confidence scoring, guided fixes) would differentiate immediately.

---

7. Sources

All information above is drawn from: Mprofit's own website (mprofit.in), Help Center (help.mprofit.in), and **Third-Party Licenses page** (self-published, legally required OSS disclosure); Google Play / App Store listings; Crunchbase, ZoomInfo, Wellfound, SoftwareSuggest, SaaSworthy, Techjockey, 360Quadrants, G2, and Trustpilot listings/reviews; public funding/press coverage (Inc42, IndianStartupNews, Entrepreneur India) for the unrelated-but-adjacent PowerUp Money data point.

No non-public systems, source code, internal APIs, or authentication-gated data were accessed in preparing this document.